

Composition of State Machines

Concurrent Composition

Modeling with State Machines

Questions:

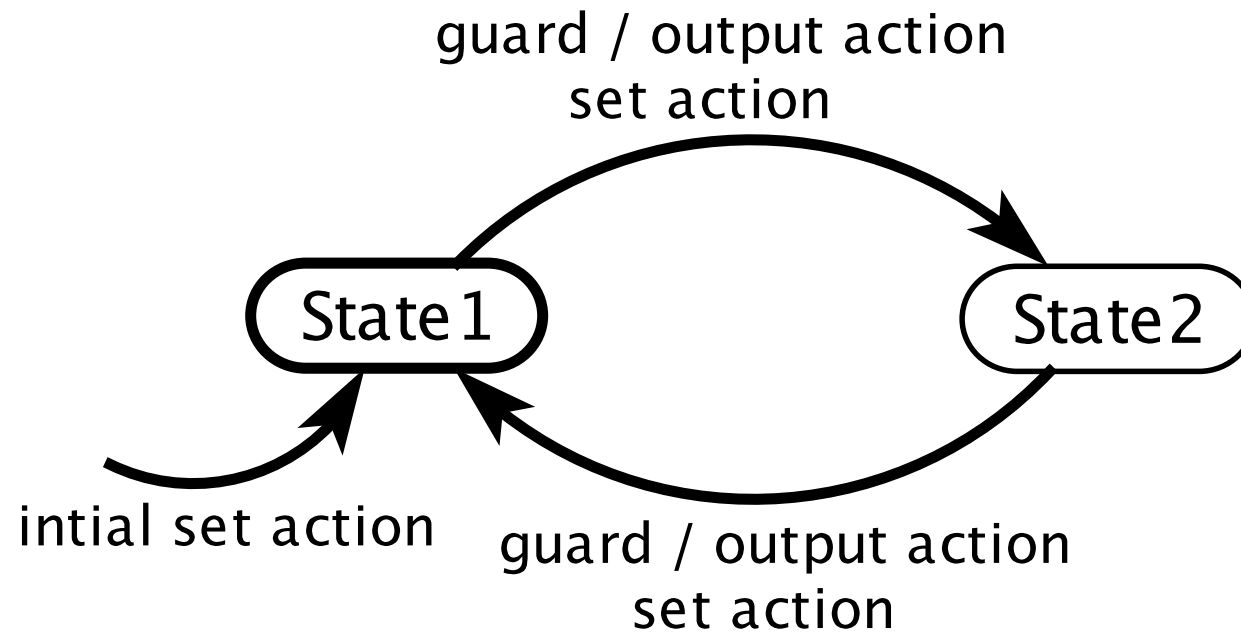
- How to represent the system for:
 - ♦ Systematic analysis
 - ♦ So that a computer program can manipulate it
- How to model its environment
- How to compose subsystems to make bigger systems
- How to check whether the system satisfies its specification in its operating environment

Notation for Extended State Machines

variable declaration(s)

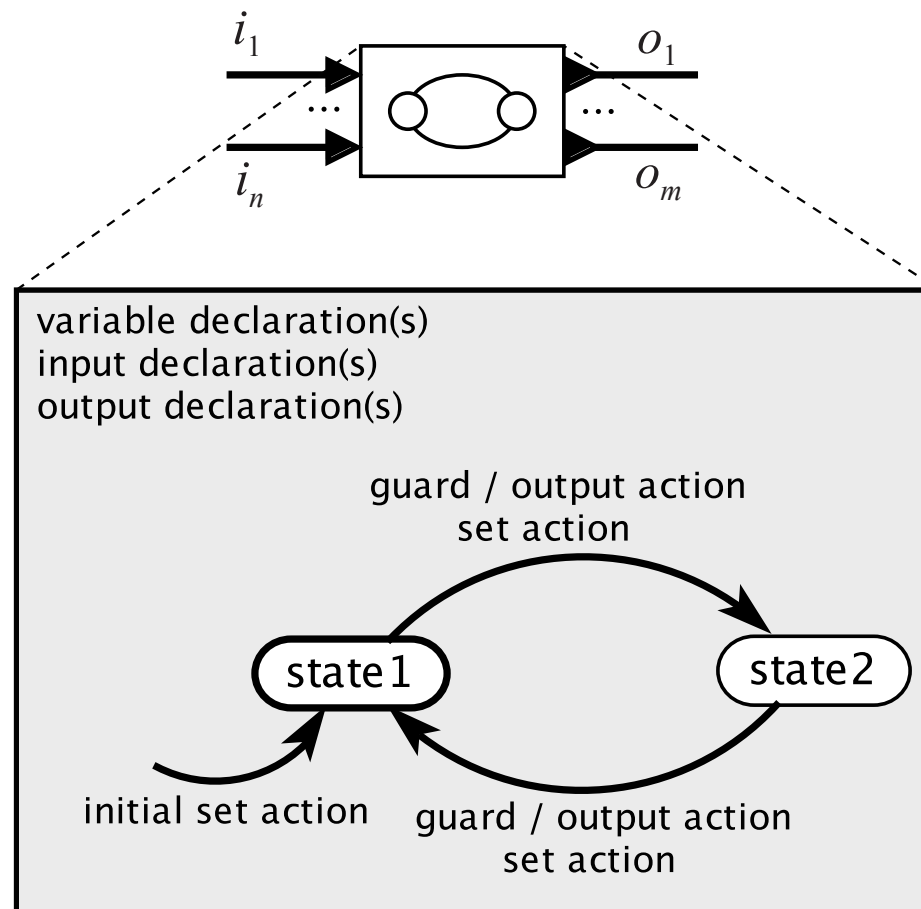
input declaration(s)

output declaration(s)



Actor Model for State Machines

Expose inputs and outputs, enabling composition:



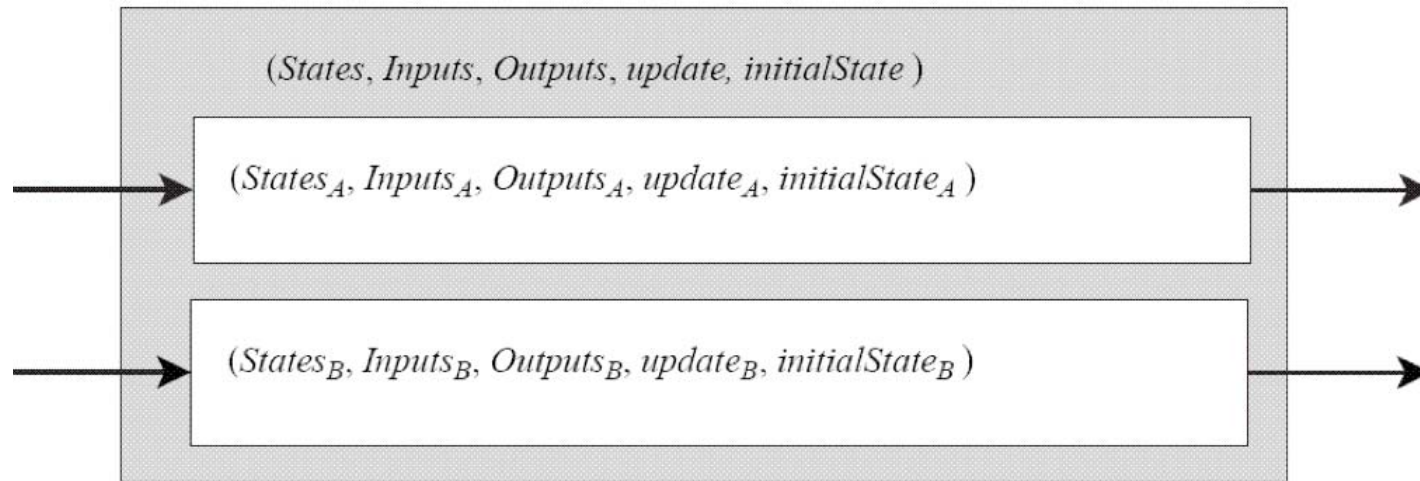
Composition of State Machines(1)

- concurrent composition
 - ◆ Side-by-side composition
 - ◆ shared variables
 - ◆ Cascade composition
 - ◆ General composition
- hierarchical composition

Composition of State Machines(2)

- concurrent composition
- hierarchical composition
 - ◆ reset transition
 - ◆ preemptive transitive
 - ◆ history transition

Side-by-Side Composition



A key question: When do these machines react?

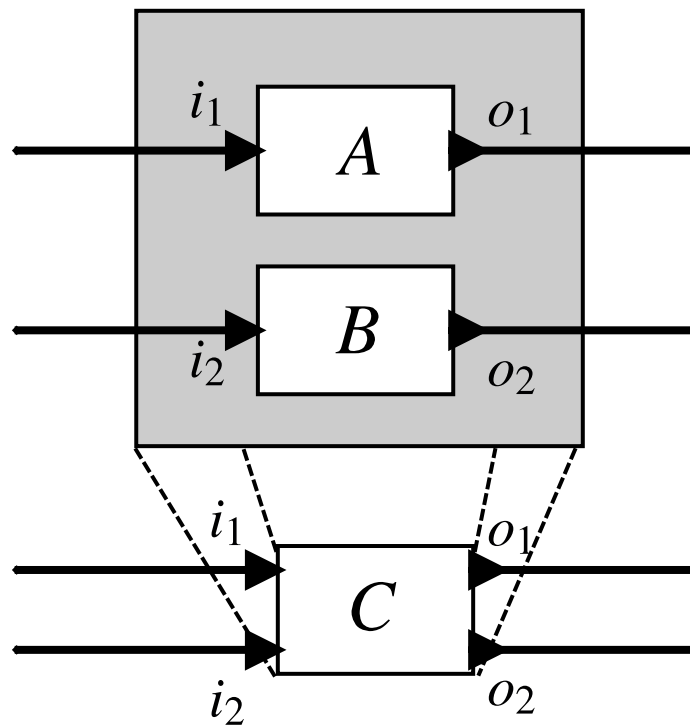
Two possibilities:

- Together (synchronous composition)
- Independently (asynchronous composition)

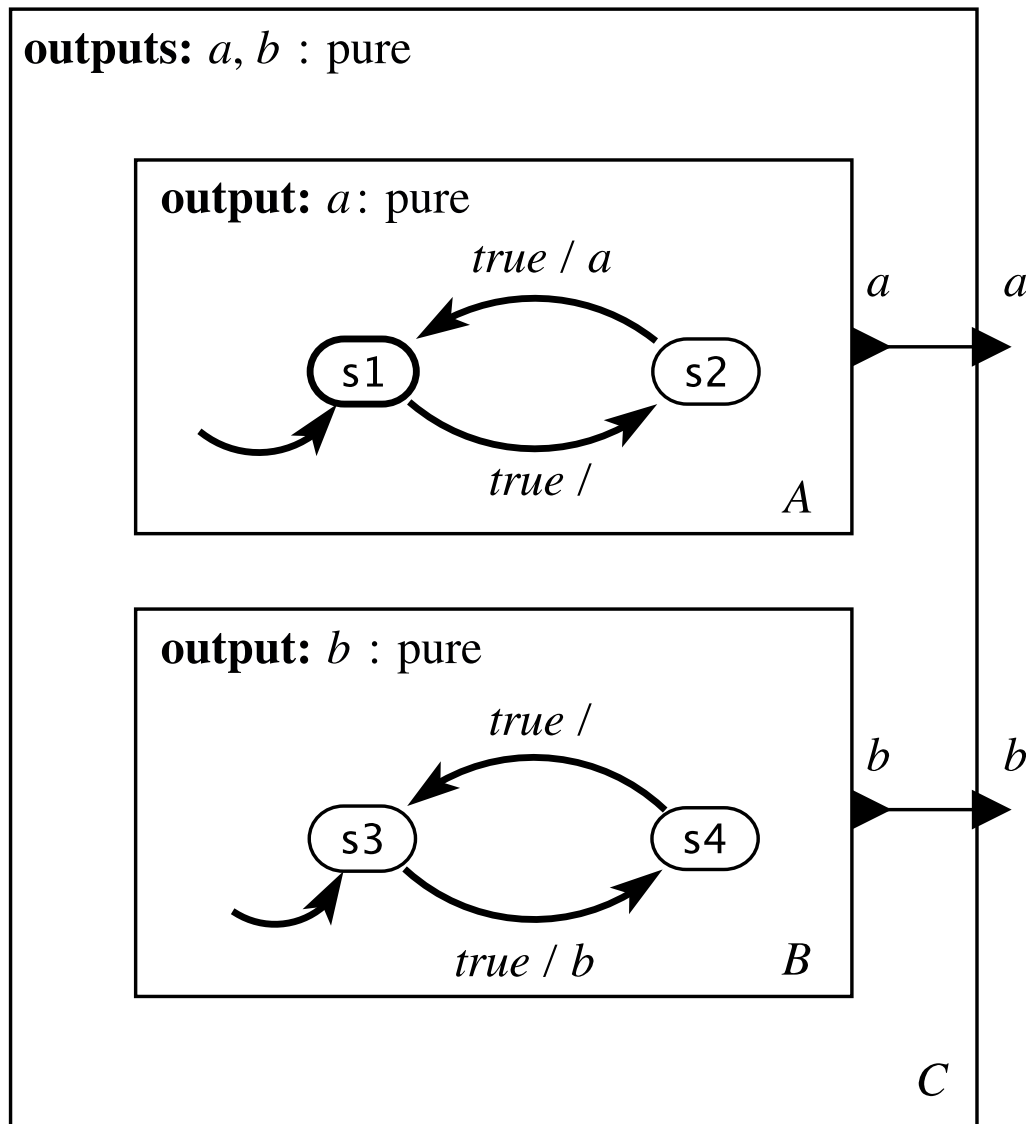
Side-by-side Synchronous Composition

- the inputs and outputs of the two actors are disjoint
- the state machines do not communicate.

Side-by-side Synchronous Composition



Example



Side-by-Side Synchronous Composition

$A = (StatesA, InputsA, OutputsA, updateA, initialStateA)$

$B = (StatesB, InputsB, OutputsB, updateB, initialStateB)$

the synchronous side-by-side composition C is

$C = (StatesC, InputsC, OutputsC, updateC, initialStateC)$

where

$StatesC = StatesA \times StatesB$

$InputsC = InputsA \times InputsB$

$OutputsC = OutputsA \times OutputsB$

$initialStateC = (initialStateA, initialStateB)$

Side-by-Side Synchronous Composition

$$\textit{update}_C((s_A, s_B), (i_A, i_B)) = ((s'_A, s'_B), (o_A, o_B)),$$

$$(s'_A, o_A) = \textit{update}_A(s_A, i_A),$$

$$(s'_B, o_B) = \textit{update}_B(s_B, i_B),$$

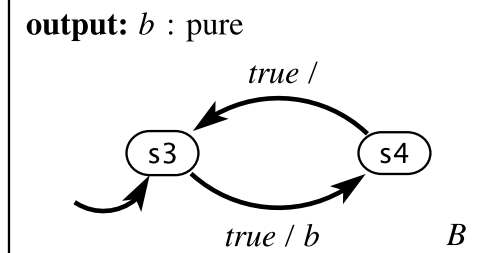
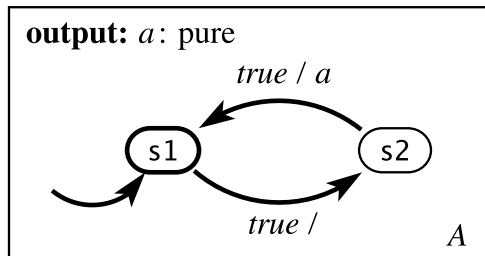
for all $s_A \in \textit{StatesA}$, $s_B \in \textit{StatesB}$, $i_A \in \textit{InputsA}$,
and $i_B \in \textit{InputsB}$.

side-by-side synchronous composition

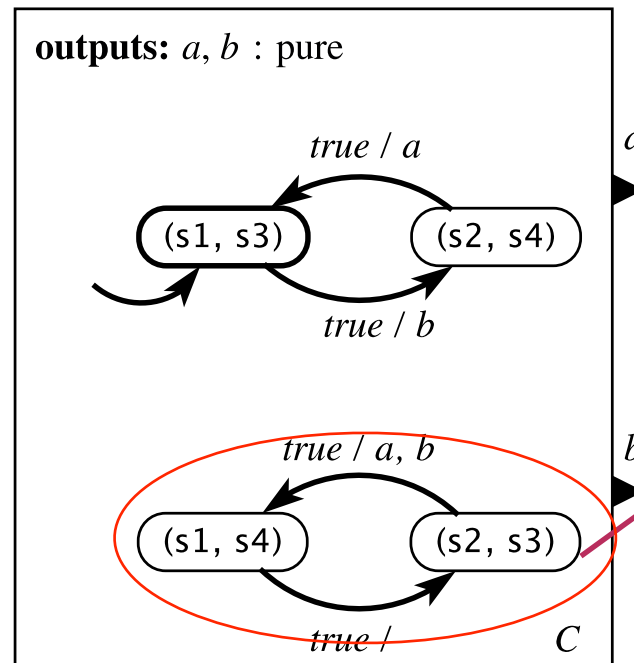
$$S_C = S_A \times S_B$$



outputs: a, b : pure



outputs: a, b : pure



Note that these two states are not *reachable*.

Synchronous composition

Side-by-side Synchronous Composition

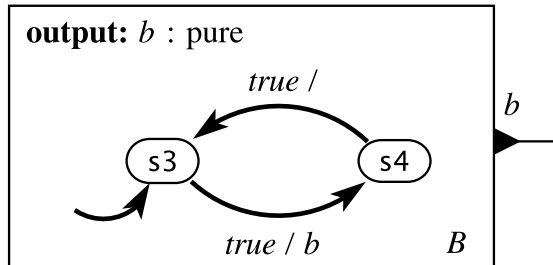
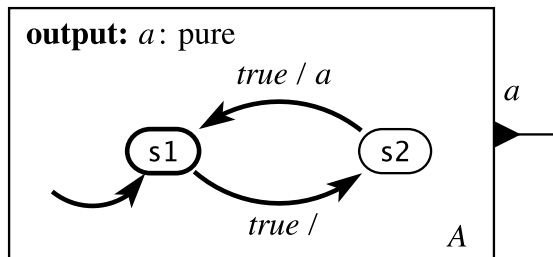
- variables are also disjoint for **extended state machines**
- If the two state machines A and B are **determinate**, then the synchronous side-by-side composition is also determinate. ----
determinacy is a **compositional** property
- **compositional property**: if a property held by the components is also a property of composition.

side-by-side Asynchronous composition

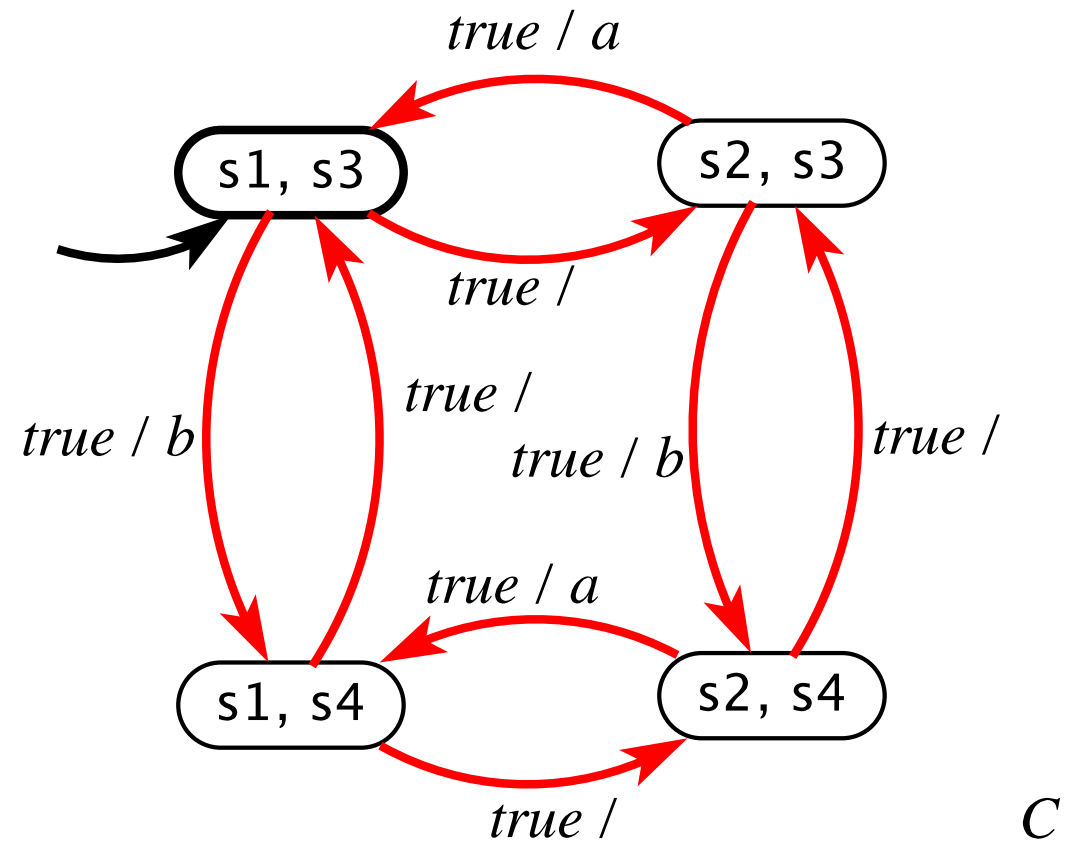
- two semantics
 - ◆ A reaction of C is a reaction of one of A or B, where the choice is nondeterministic-- interleaving semantics.
 - ◆ A reaction of C is a reaction of A, B or both A and B, where the choice is nondeterministic.

Example for semantics I

outputs: a, b : pure



outputs: a, b : pure



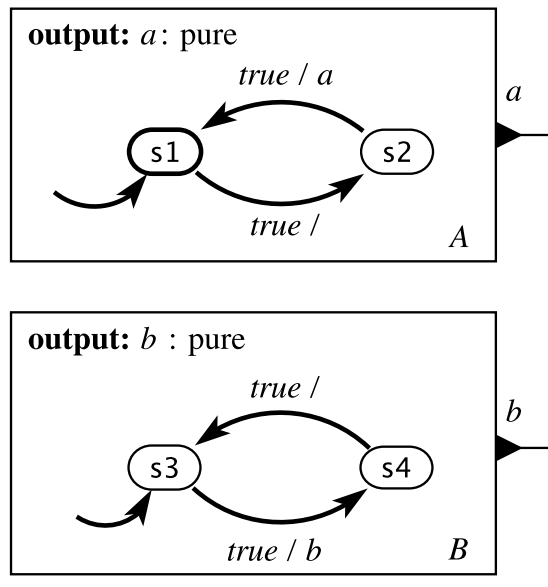
side-by-side Asynchronous composition---semantics 1

- the symbolic definition of C has the same definitions of $StatesC$, $InputsC$, $OutputsC$, and $initialStateC$ as for synchronous composition
- update function:

$update_C((s_A, s_B), (i_A, i_B)) = ((s'_A, s'_B), (o_A, o_B)),$
 $(s'_A, o_A) = update_A(s_A, i_A)$ and $s'_B = s_B, o_B = absent$ or
 $(s'_B, o_B) = update_B(s_B, i_B)$, and $s'_A = s_A, o_A = absent$
 for all $s_A \in StatesA$, $s_B \in StatesB$, $i_A \in InputsA$, and $i_B \in InputsB$.

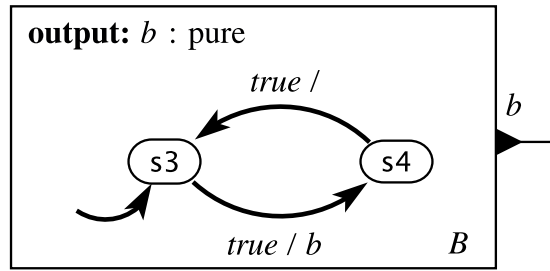
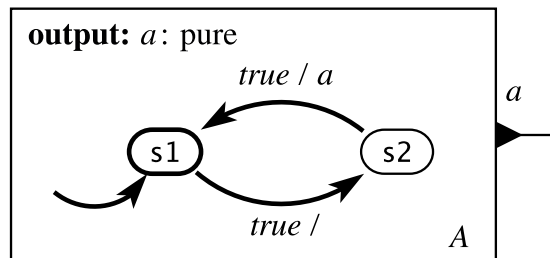
synchronous vs. asynchronous

outputs: a, b : pure

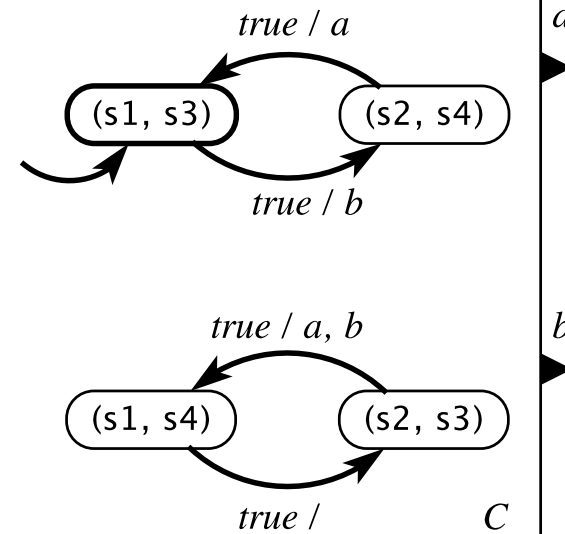


synchronous composition

outputs: a, b : pure

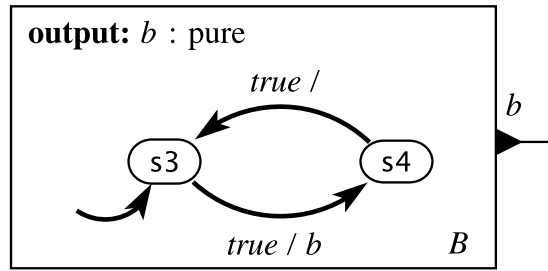
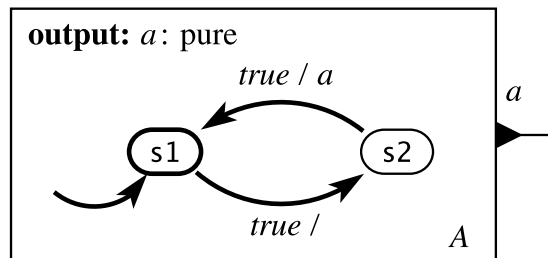


outputs: a, b : pure

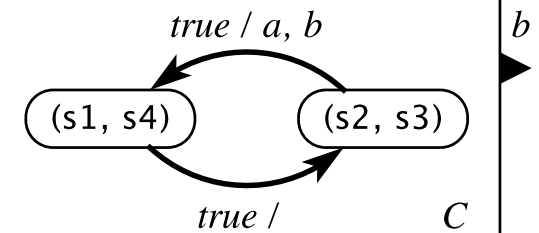
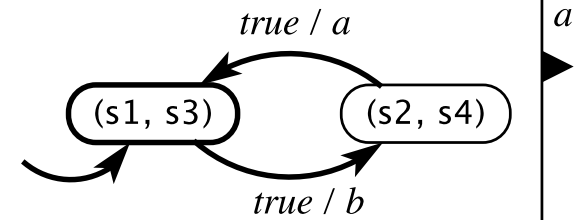


synchronous composition

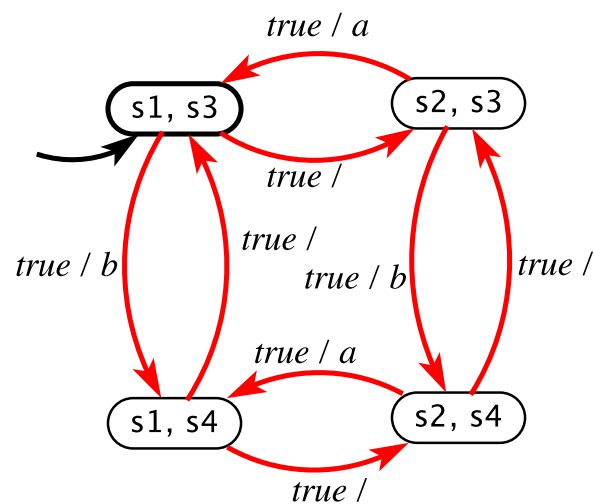
outputs: a, b : pure



outputs: a, b : pure



outputs: a, b : pure



asynchronous composition for semantics I

share variable

- An **extended state machine** has local variables that can be read and written as part of taking a transition.
- allow these variables to be shared among a group of machines.
- ensure the semantics of the compositional models

synchronous of shared variables

- a fixed order determined

asynchronous example: two servers

- two servers receive requests from a network
- the servers share a queue of requests
- If one server is busy, the other server can respond to a request,
- a shared variable *pending* counts the number of pending job requests.
- assuming asynchronous composition under semantics 1.

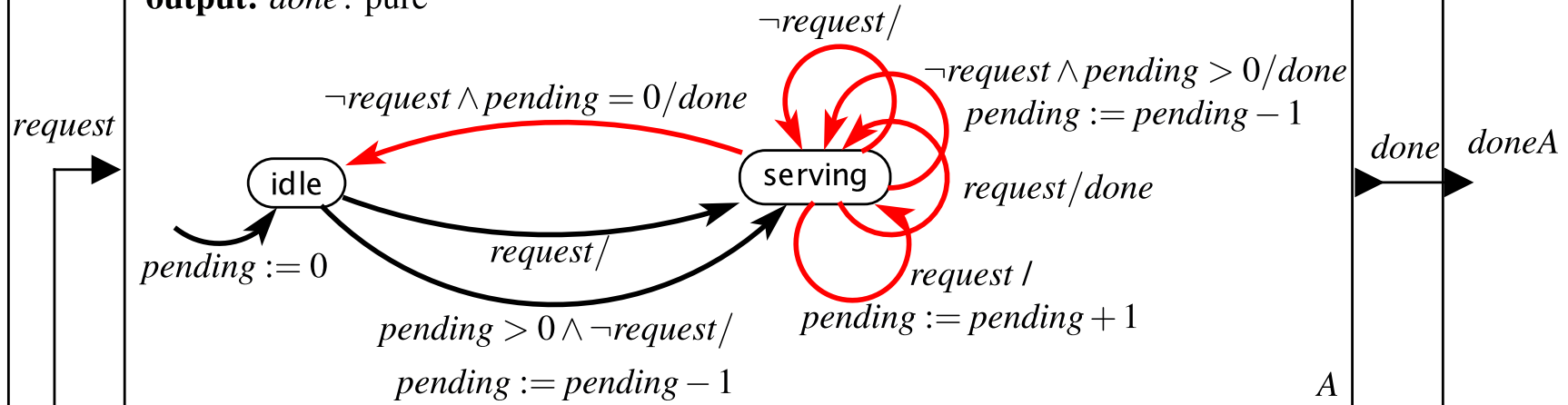
shared variable: *pending*: int

input: *request*: pure

outputs: *doneA*, *doneB* : pure

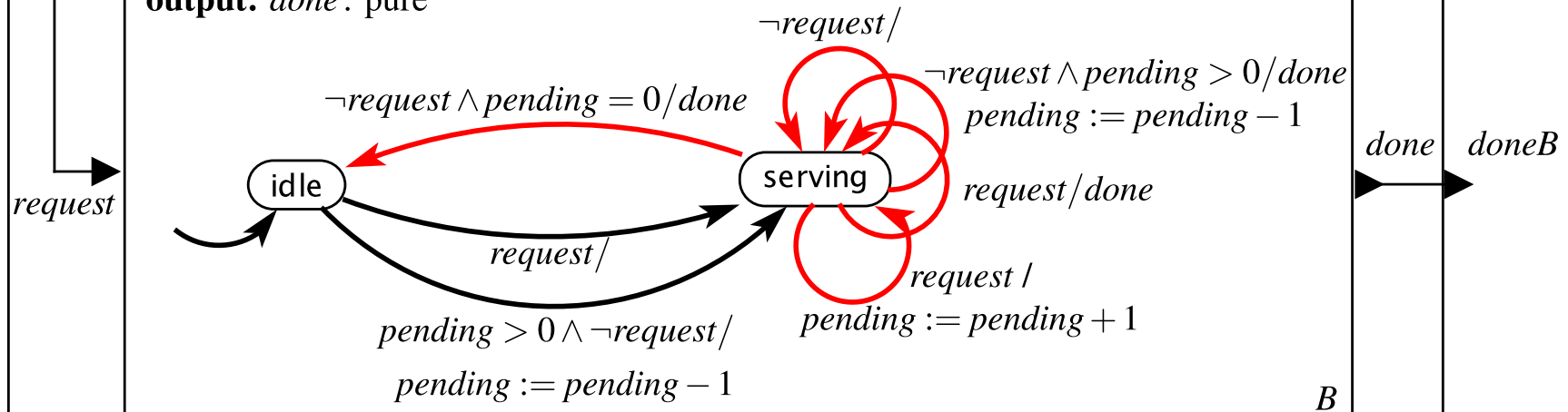
input: *request*: pure

output: *done*: pure



input: *request*: pure

output: *done*: pure

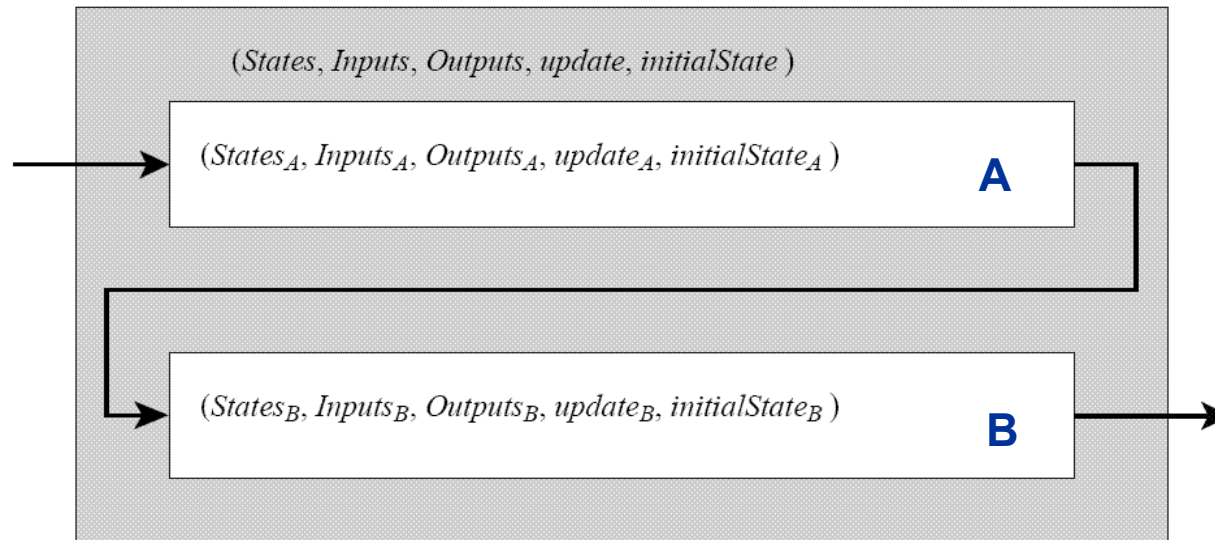


C

Analysis this servers example

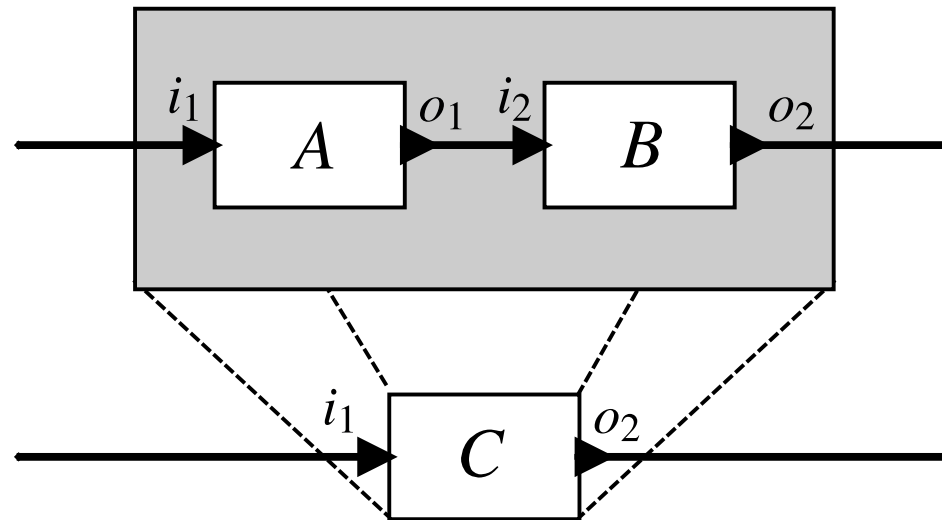
- in the **interleaving semantics**, access to the shared variable are **atomic operations**,
- the choice of semantics 1 is reasonable
- this semantics would not work if the two machines had independent inputs, maybe lost requests.
- The difficulties of shared variables reflect the inherent complexity of concurrency models with shared variables.

Cascade Composition



- The output of machine A feeds the input of B.
- This style of composition is called **cascade composition** or serial composition.

Cascade Composition

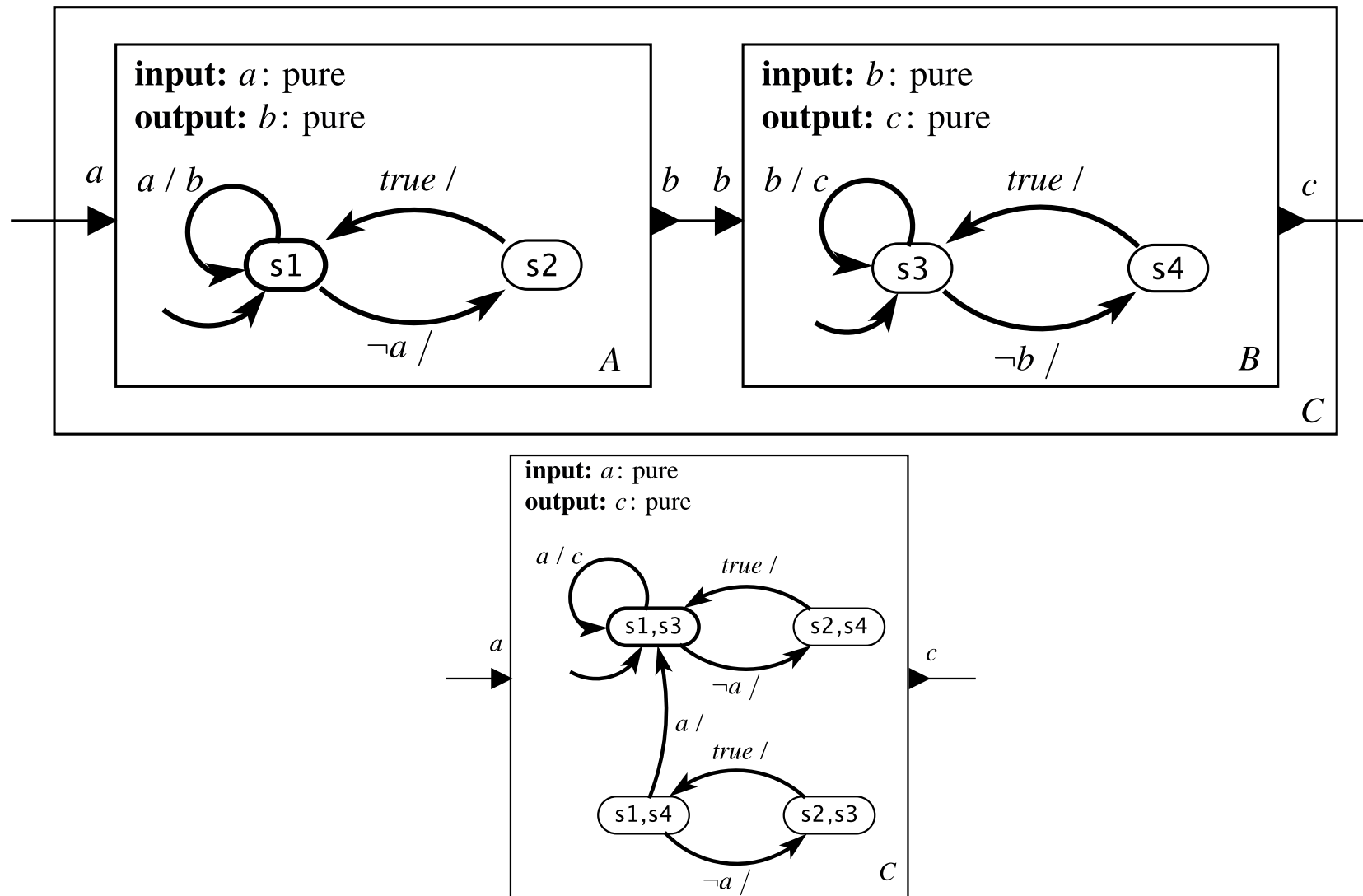


- $V_1 \subseteq V_2$
- This asserts that any output produced by A on port o_1 is an acceptable input to B on port i_2 .
- The composition **type checks**.

Cascade Composition

- the asynchronous composition
 - ✦ introduce some machinery for buffering the data that is sent from A to B.
- synchronous composition
 - ✦ a reaction of C consists of a reaction of both A and B, where A reacts first, produces its output (if any), and then B reacts.
 - ✦ They are simultaneous.

example

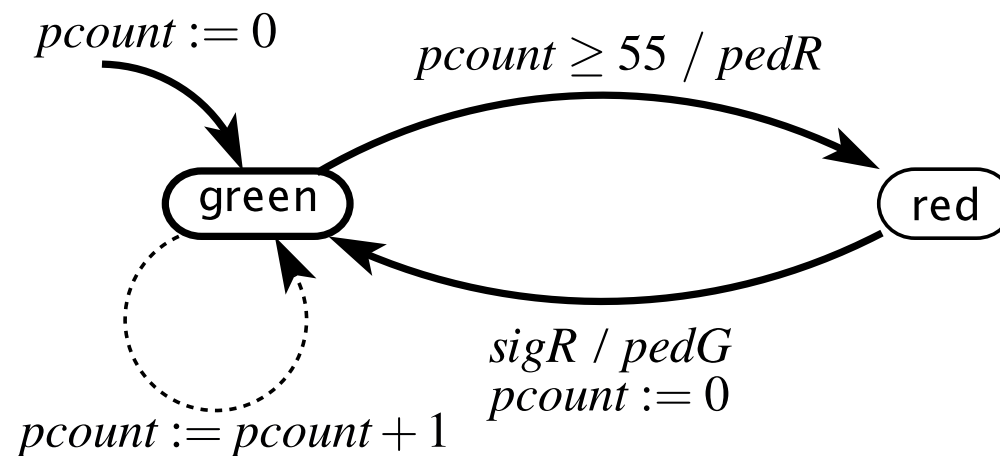


Example: Pedestrian Light

variable: $pcount: \{0, \dots, 55\}$

input: $sigR$: pure

outputs: $pedG, pedR$: pure



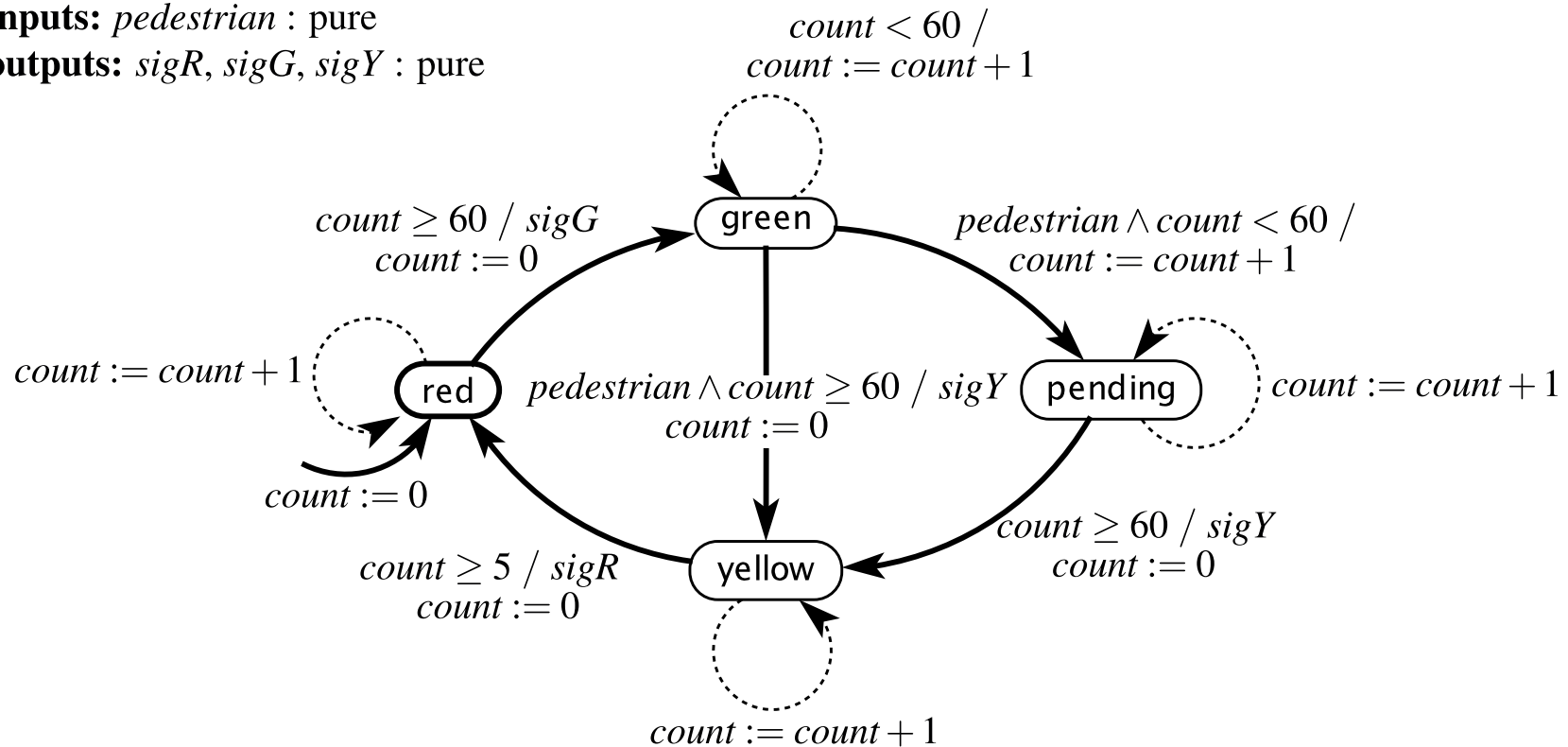
- *This light stays green for 55 seconds, then goes red. Upon receiving a $sigR$ input, it repeats the cycle.*

Example: Cross Light

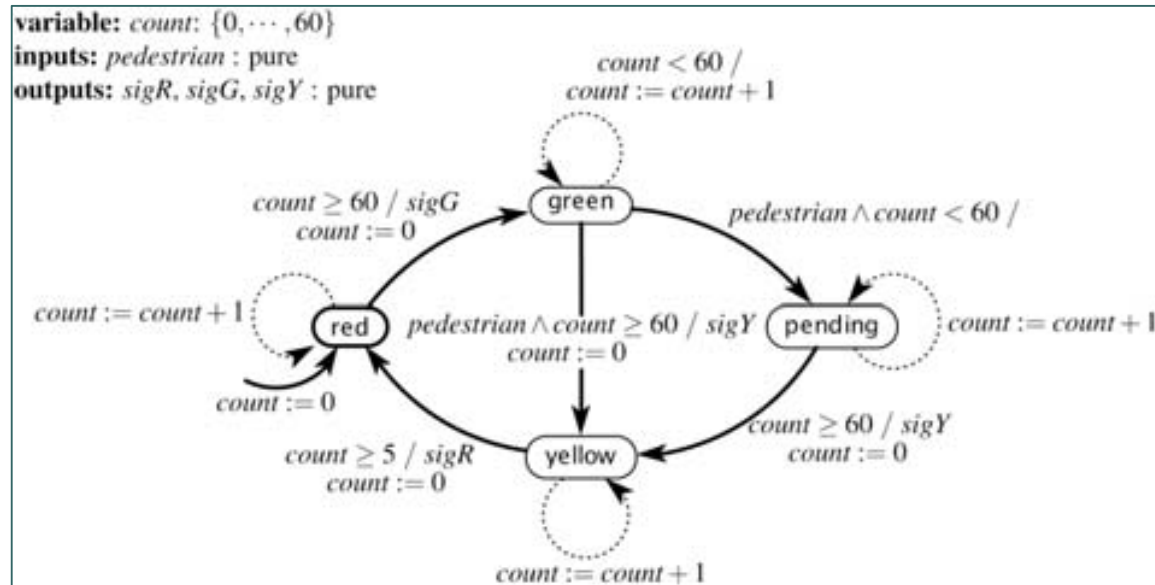
variable: $count: \{0, \dots, 60\}$

inputs: $pedestrian : \text{pure}$

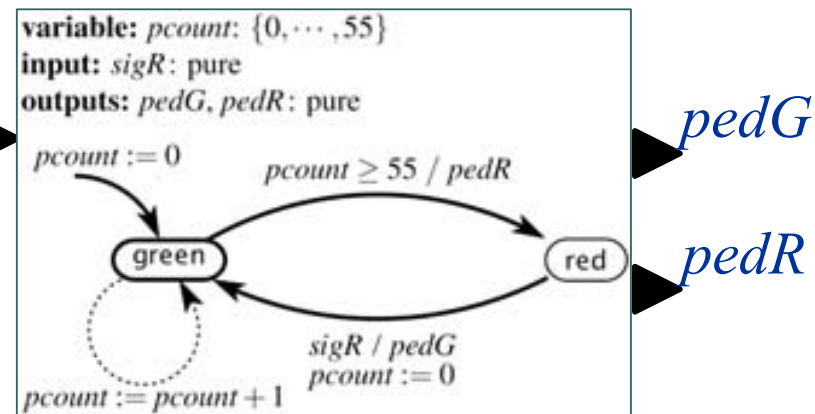
outputs: $sigR, sigG, sigY : \text{pure}$



Pedestrian Light with Car Light

*sigY**sigG**sigR*

*What is the size of
the state space of the
composite machine?*

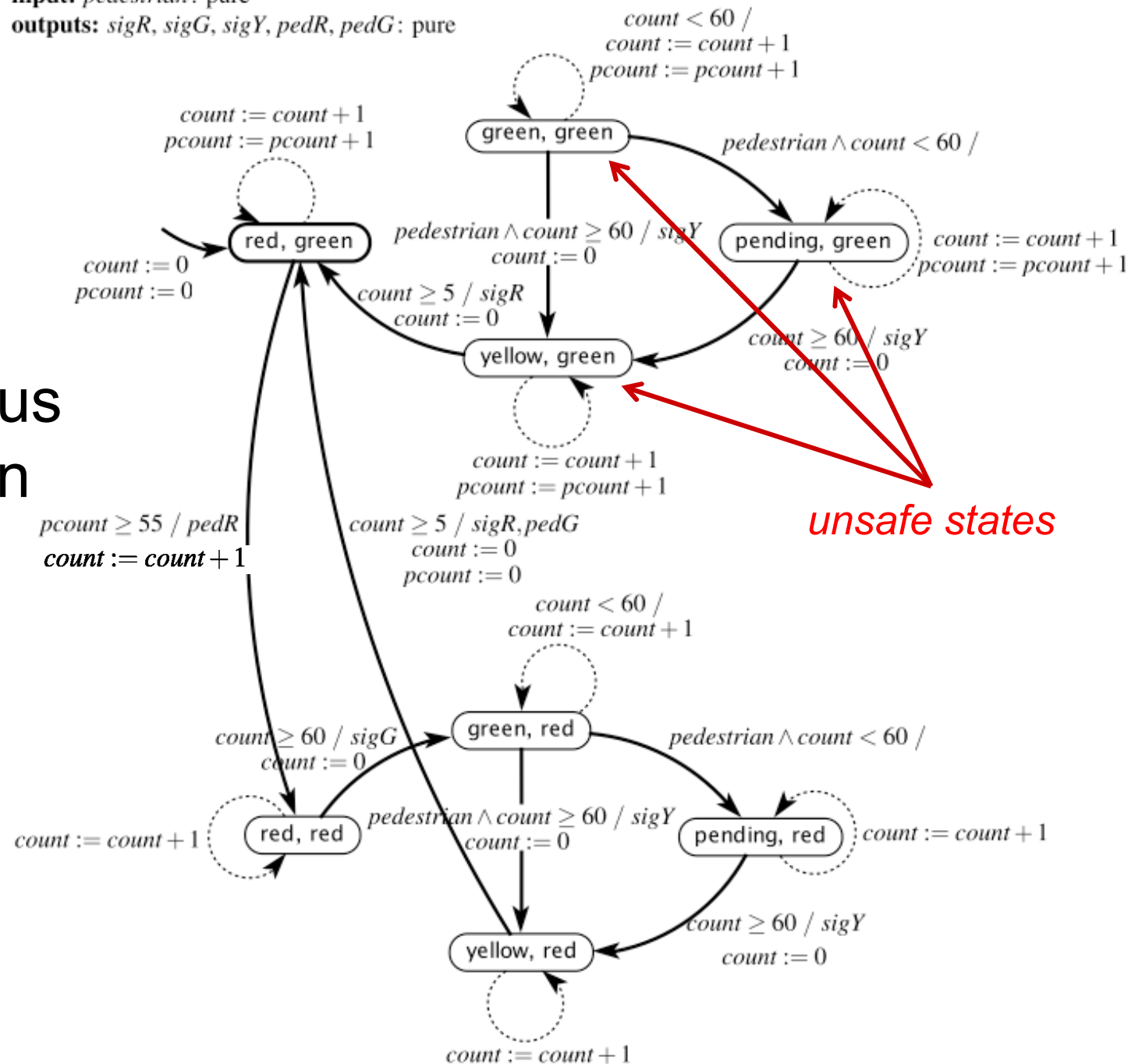
sigR*pedG**pedR*

variables: $count: \{0, \dots, 60\}, pcount: \{0, \dots, 55\}$

input: $pedestrian: \text{pure}$

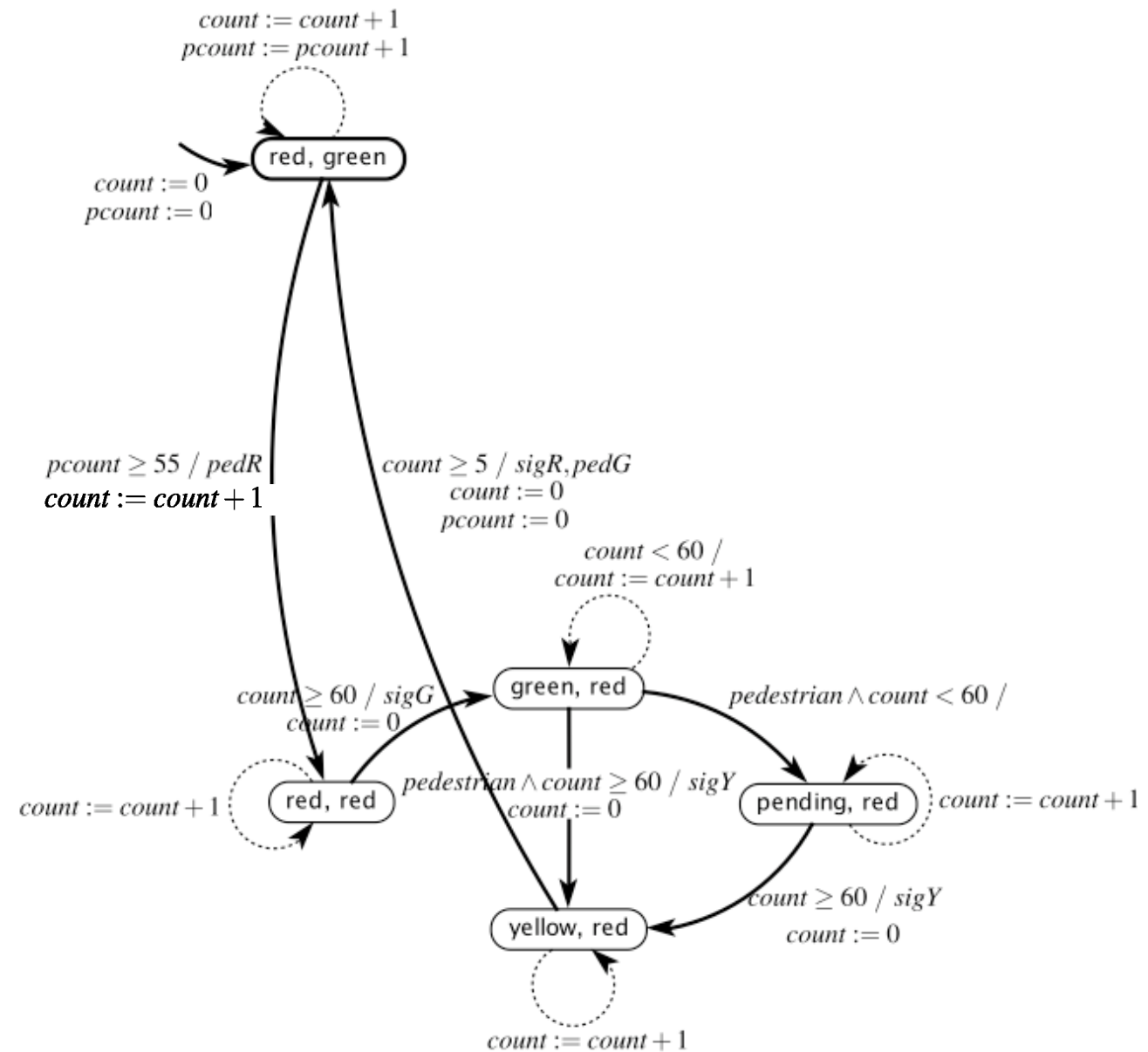
outputs: $sigR, sigG, sigY, pedR, pedG: \text{pure}$

Synchronous composition

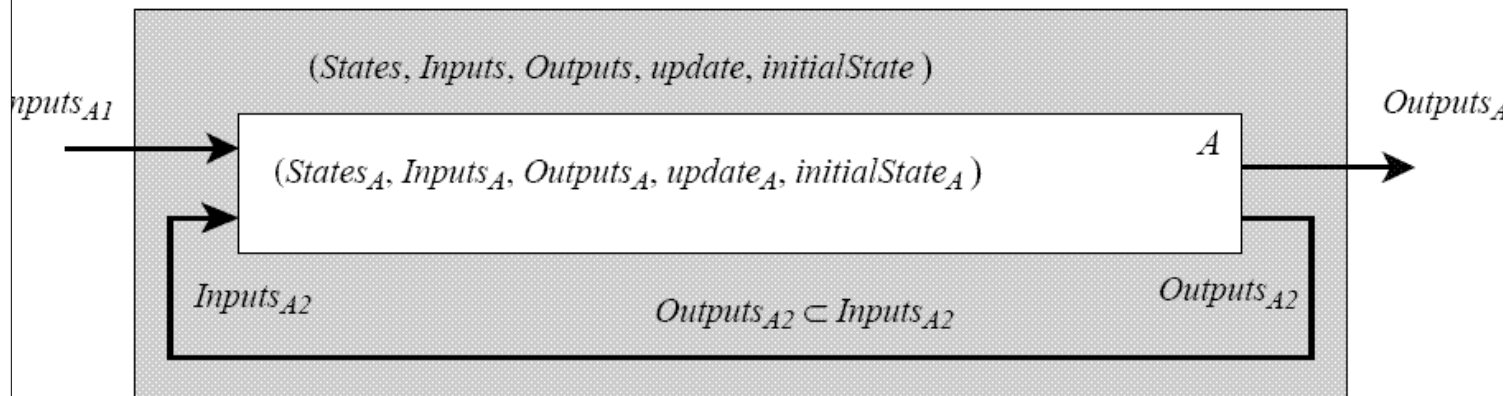


variables: $count: \{0, \dots, 60\}, pcount: \{0, \dots, 55\}$
input: $pedestrian: \text{pure}$
outputs: $sigR, sigG, sigY, pedR, pedG: \text{pure}$

Synchronous
composition
with
unreachable
states
removed



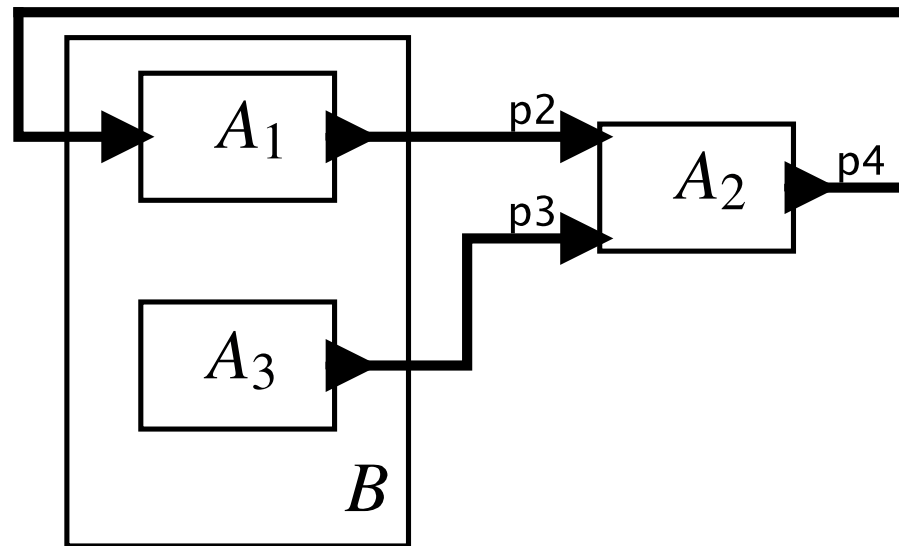
Feedback Composition



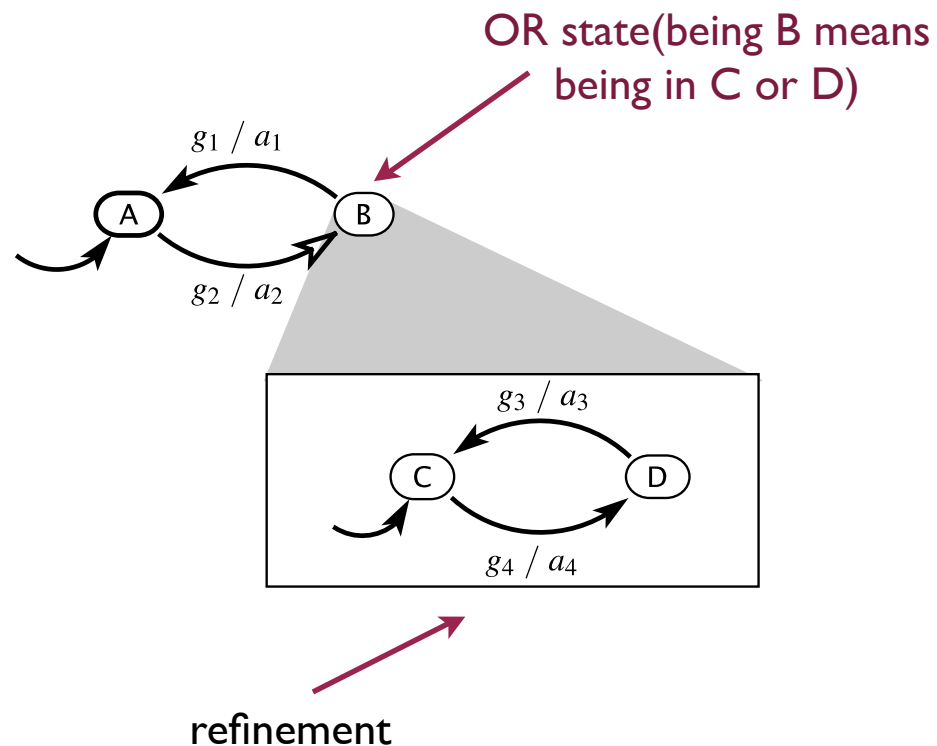
- *More on this later... Very subtle.*
 - ✦ which machine should react first, $A1$ or $A2$?

General Composition

- Side-by-side and cascade composition provide the basic building blocks for building more complex compositions of machines.



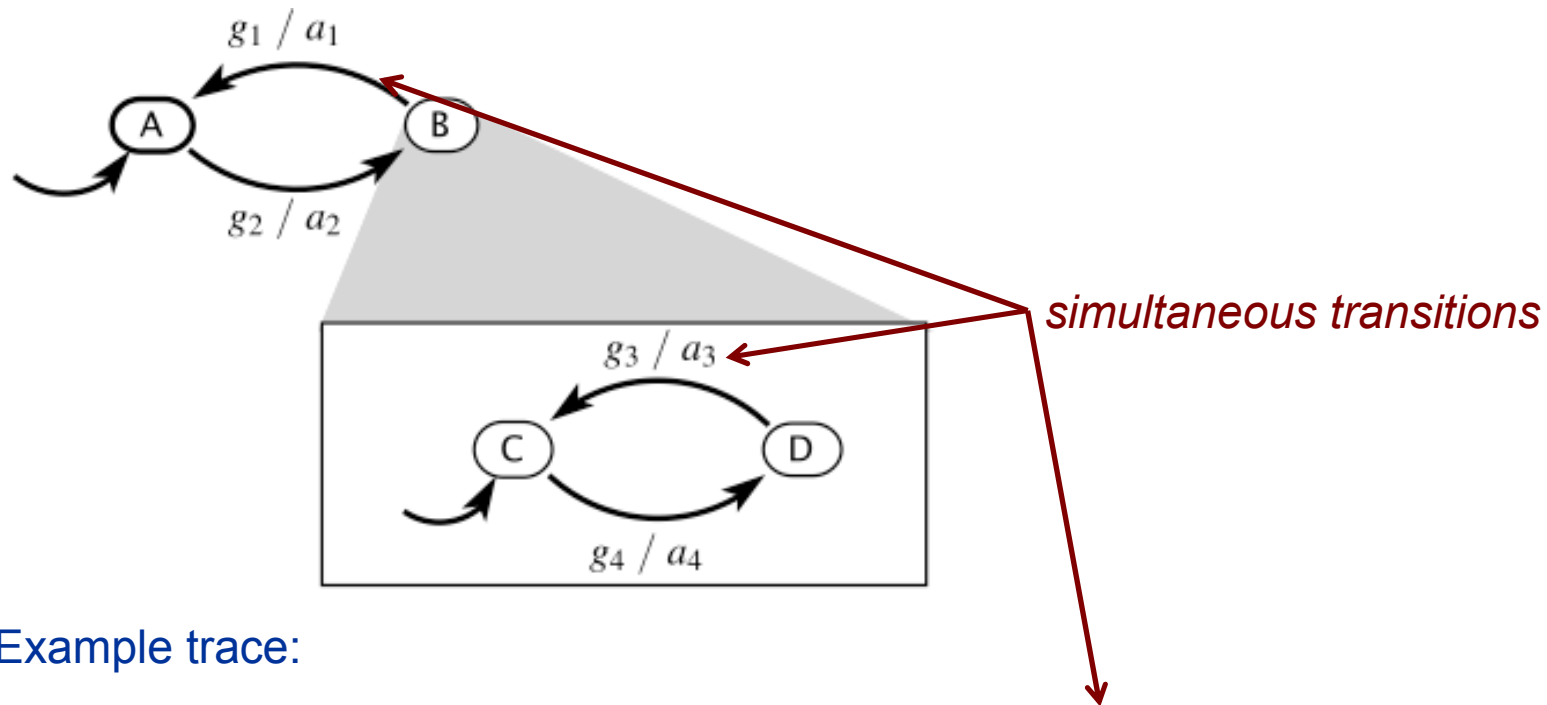
Hierarchical State Machines



Reaction:

1. the refinement of the current state (if any) reacts.
 2. the top-level machine reacts.
 3. If both produce outputs, they are required to not conflict.
- The two steps are part of the same reaction.

Hierarchical State Machines

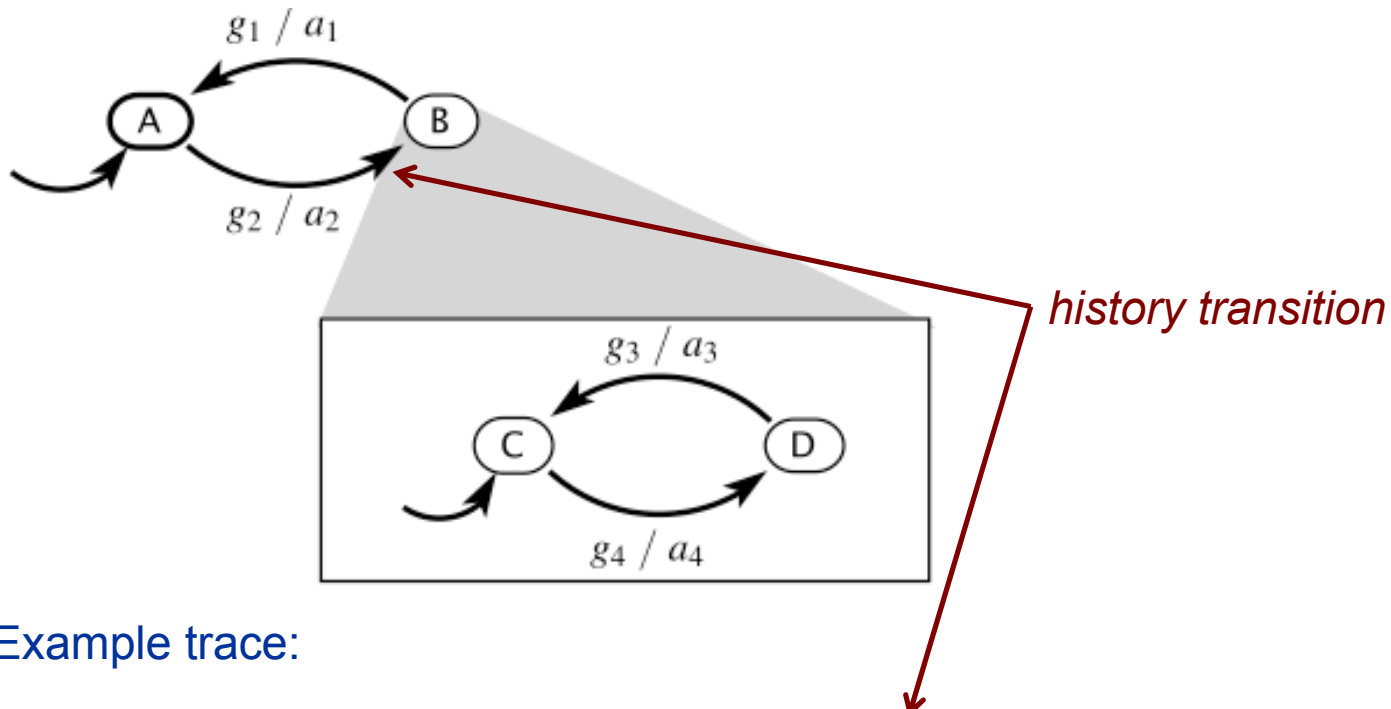


Example trace:

$$A \xrightarrow{g_2/a_2} C \xrightarrow{g_4/a_4} D \xrightarrow{g_1/a_1} A \xrightarrow{g_2/a_2} D \xrightarrow{g_3 \wedge g_1/a_3, a_1} A \dots$$

Simultaneous transitions can produce multiple outputs.
These are required to not conflict.

Hierarchical State Machines

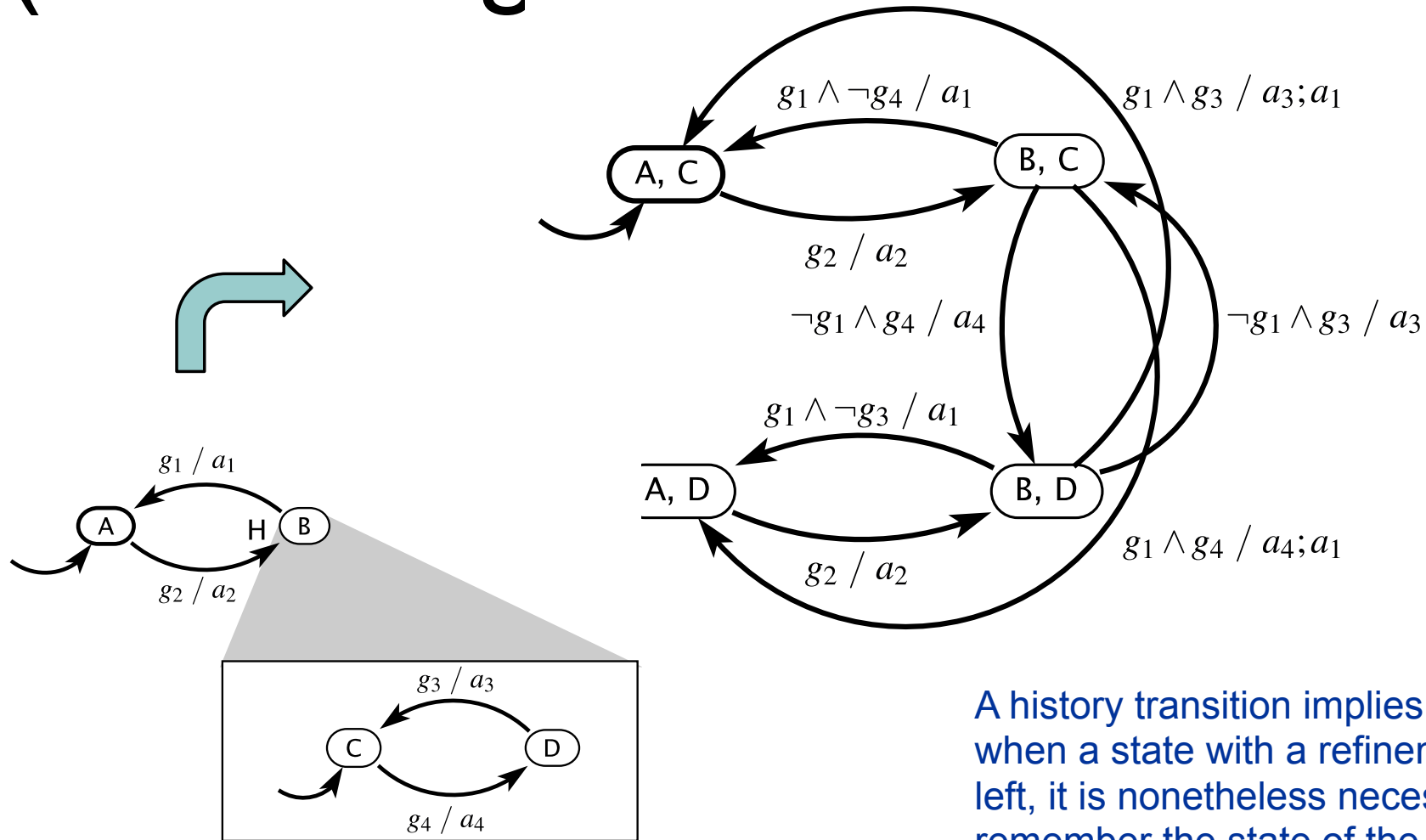


Example trace:

$$A \xrightarrow{g_2/a_2} C \xrightarrow{g_4/a_4} D \xrightarrow{g_1/a_1} A \xrightarrow{g_2/a_2} D \xrightarrow{g_3 \wedge g_1/a_3, a_1} A \dots$$

Simultaneous transitions can produce multiple outputs.
These are required to not conflict.

Flattening the state machine (assuming history transitions):

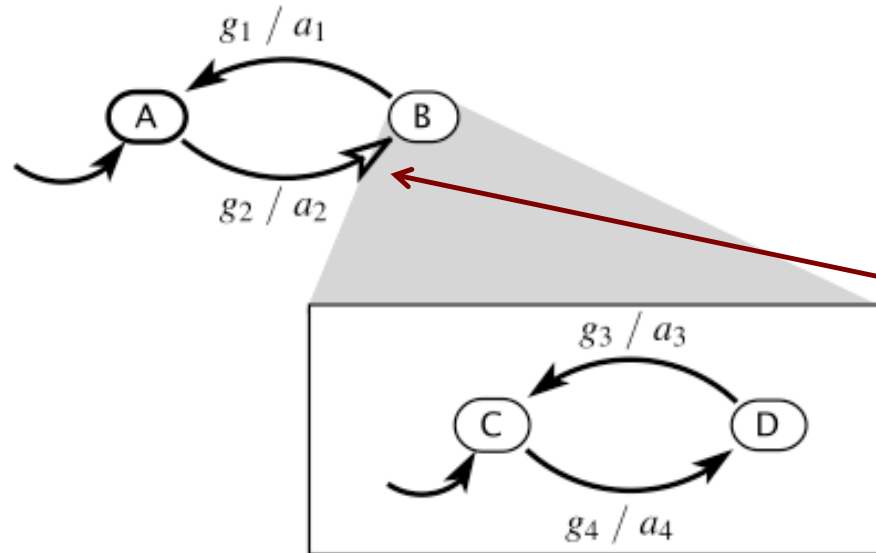


A history transition implies that when a state with a refinement is left, it is nonetheless necessary to remember the state of the refinement. Hence A, C and A, D .

Hierarchical State Machines

Reset transitions

Reset Transitions



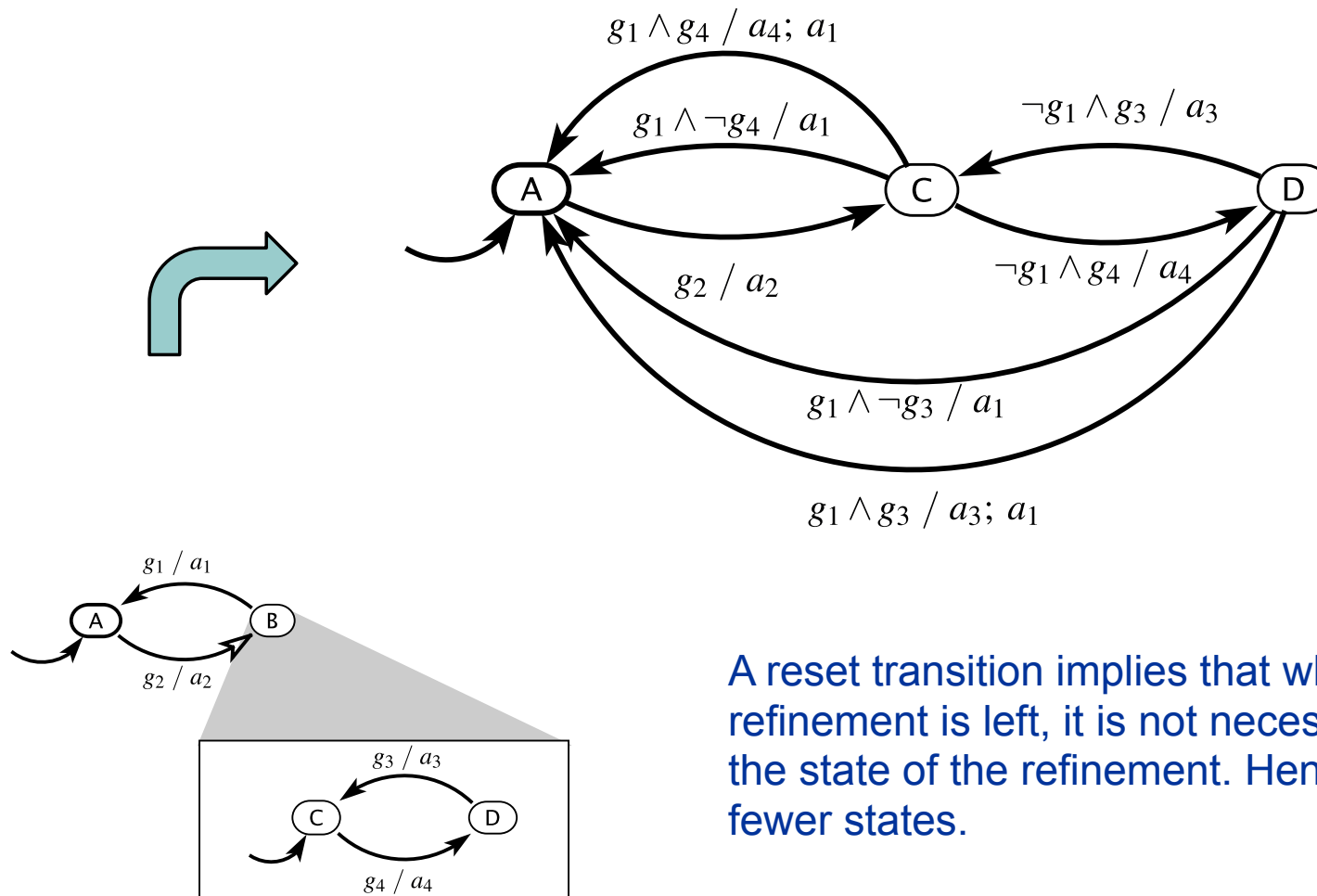
A reset transition always initializes the refinement of the destination state to its initial state.

Example trace:

$A \xrightarrow{g_2/a_2} C \xrightarrow{g_4/a_4} D \xrightarrow{g_1/a_1} A \xrightarrow{g_2/a_2} C \xrightarrow{g_4 \wedge g_1/a_4, a_1} A \dots$

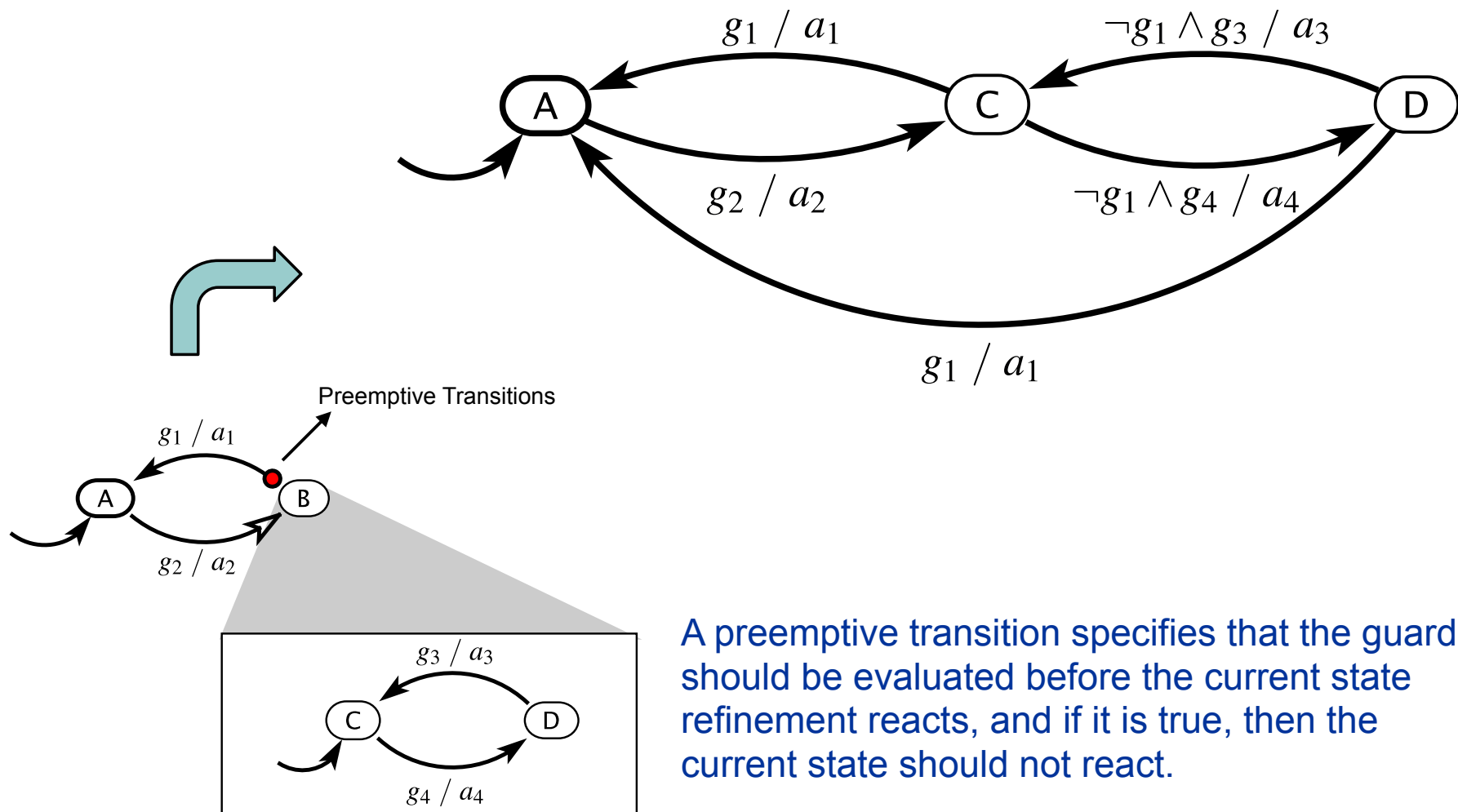
A reset transition implies that when a state with a refinement is left, you can forget the state of the refinement.

Flattening the state machine (assuming reset transitions):



A reset transition implies that when a state with a refinement is left, it is not necessary to remember the state of the refinement. Hence there are fewer states.

Preemptive Transitions



A preemptive transition specifies that the guard should be evaluated before the current state refinement reacts, and if it is true, then the current state should not react.

Summary

- Composition enables building complex systems from simpler ones.
- Hierarchical FSMs enable compact representations of complex behaviors.
- Both forms of composition can be converted to single flat FSMs, but the resulting FSMs are quite complex and difficult to analyze by hand.